



---

# Git 零基础使用教程（GitHub 仓库管理）

---

嵌入式学习必备技能



2026-6-3

非

## 目 录

|                                           |    |
|-------------------------------------------|----|
| 1 前置准备：安装 Git 与配置 SSH 密钥 .....            | 1  |
| 1.1 下载与安装 Git .....                       | 1  |
| 1.2 配置全局用户信息（仅电脑第一次使用 Git 时执行） .....      | 1  |
| 1.3 生成 SSH 密钥对 .....                      | 1  |
| 1.4 将公钥添加到 GitHub.....                    | 2  |
| 1.5 测试 SSH 连接 .....                       | 2  |
| 2 解决网络问题：SSH 端口 22 改为 443.....            | 3  |
| 2.1 修改 SSH 配置文件 .....                     | 3  |
| 2.1.1 Windows 系统： .....                   | 3  |
| 2.1.2 Linux/macOS 系统： .....               | 3  |
| 2.2 验证配置是否生效.....                         | 4  |
| 2.3 （可选）配置 Git 全局使用 443 端口 .....          | 4  |
| 2.4 SSH 与 HTTPS 地址切换 .....                | 4  |
| 3 GitHub 仓库共享与拉取.....                     | 6  |
| 3.1 如何共享仓库给其他人（添加协作者） .....               | 6  |
| 3.2 克隆（拉取）别人的仓库到本地——如何指定存放路径.....         | 6  |
| 4 全新文件夹首次上传 GitHub 完整流程（使用 main 分支） ..... | 10 |
| 4.1 GitHub 网页端新建空仓库.....                  | 10 |
| 4.2 本地初始化与首次上传.....                       | 10 |
| 4.3 分支搞错了怎么办？（master→main 迁移） .....       | 13 |
| 5 日常更新简化流程（三步走） .....                     | 16 |
| 5.1 Git 三大区域.....                         | 16 |
| 5.2 文件流转.....                             | 16 |
| 5.3 删除文件/文件夹的正确操作 .....                   | 17 |
| 6 Git 常用操作命令速查.....                       | 19 |
| 6.1 查看类命令.....                            | 19 |
| 6.2 分支管理命令.....                           | 19 |
| 6.3 远程仓库操作.....                           | 20 |

|                                             |    |
|---------------------------------------------|----|
| 6.4 版本回退.....                               | 20 |
| 6.5 临时保存 (stash) .....                      | 20 |
| 6.6 文件操作.....                               | 21 |
| 7 常见问题与避坑指南.....                            | 22 |
| 7.1 推送报错：文件超过 100MB .....                   | 22 |
| 7.2 推送报错：Connection timed out port 22 ..... | 22 |
| 7.3 不要嵌套初始化 Git 仓库.....                     | 22 |
| 7.4 提交备注规范.....                             | 22 |
| 7.5 .gitignore 不生效怎么办? .....                | 22 |
| 7.6 多人协作时严禁使用--force 强制推送 .....             | 23 |
| 7.7 提交备注写错了怎么改? .....                       | 24 |

## 1 前置准备：安装 Git 与配置 SSH 密钥

Git 是本地电脑安装的分布式版本控制工具，不需要联网就能运行，项目里隐藏文件夹.git 就是本地仓库，负责记录代码每一次修改、保存全量历史版本，支持分支切换、版本回退、代码对比。

### 1.1 下载与安装 Git

(1)访问 Git 官网：<https://git-scm.com/downloads>

(2)下载对应操作系统（Windows/macOS/Linux）的安装包

(3)按默认选项（安装 d 盘的文件夹里面，其他配置一律默认）完成安装，安装完成后在任意文件夹右键可以看到 GitBashHere 选项

### 1.2 配置全局用户信息（仅电脑第一次使用 Git 时执行）

打开 Git Bash，输入以下命令（桌面打开 bash 图标配置，不要进任何文件夹打开，这是在配置全局）：

表 1-1 初始化配置全局

|                                                 |
|-------------------------------------------------|
| bash                                            |
| git config --global user.name "你的 GitHub 用户名"   |
| git config --global user.email "你的 GitHub 注册邮箱" |

### 1.3 生成 SSH 密钥对

SSH 密钥用于安全连接 GitHub，避免每次操作都要输入账号密码。

在 Git Bash 中执行以下命令生成密钥：

表 1-2 配置 SSH 密钥

|                                           |
|-------------------------------------------|
| bash                                      |
| ssh-keygen -t ed25519 -C "你的 GitHub 注册邮箱" |

如果系统不支持 ed25519 算法，可改用 RSA（基本都是第一种方法）：

表 1-3 使用 RSA 配置 SSH 密钥

|      |
|------|
| bash |
|------|

```
ssh-keygen -t rsa -b 4096 -C "你的 GitHub 注册邮箱"
```

连续按回车确认默认保存位置和空密码即可。生成后会在 ~/.ssh/ 目录下得到两个文件：

`id_ed25519`——私钥（保密，不要泄露）

`id_ed25519.pub`——公钥（需要添加到 GitHub）

## 1.4 将公钥添加到 GitHub

查看并复制公钥内容：

表 1-4 查看复制公钥内容

|                           |
|---------------------------|
| bash                      |
| cat ~/.ssh/id_ed25519.pub |

选中输出的全部内容，复制到剪贴板。

登录 GitHub，点击右上角头像→Settings→左侧边栏选择 SSHandGPGkeys。

点击 NewSSH key，在 Title 字段填写一个描述性名称（如"MyPC"），在 Key 字段粘贴刚才复制的公钥内容。

点击 Add SSH key 保存。

## 1.5 测试 SSH 连接

表 1-5 测试 SSH 配置是否成功

|                       |
|-----------------------|
| bash                  |
| ssh -T git@github.com |

如果配置成功，会看到类似以下输出：

text

Hi 你的用户名! You've successfully authenticated, but GitHub does not provide shell access.

## 2 解决网络问题：SSH 端口 22 改为 443

在国内公司、校园等网络环境下，SSH 默认的 22 端口经常被防火墙封锁，导致连接超时。GitHub 提供了备用的 443 端口（HTTPS 端口，几乎所有网络都开放）来解决这个问题。

克隆、推送代码有两种仓库地址，适用场景完全不同：

（1）HTTPS 地址（格式：`https://github.com/用户名/仓库名.git`）

认证：每次 push/pull 需要输入账号 Token，无需配置密钥；

网络：不受 22 端口限制，校园网、公司内网基本都能连通；

适用：临时拉取开源代码、不想配置 SSH 密钥的场景。

（2）SSH 地址（格式：`git@github.com:用户名/仓库名.git`）

认证：本地配置一次 SSH 密钥，后续永久免密推送拉取；

网络：默认使用 22 端口，国内多数内网会封禁 22 端口，超时可切换 443 端口；

适用：长期个人开发、频繁提交代码，推荐嵌入式开发者使用。

### 2.1 修改 SSH 配置文件

#### 2.1.1 Windows 系统：

进入用户目录下的.ssh 文件夹：`C:\Users\你的用户名\.ssh\`

如果不存在 config 文件，新建一个（无扩展名）

用记事本打开 config，添加以下内容：

表 2-1 修改 SSH 使用 443 端口

|                                                                    |
|--------------------------------------------------------------------|
| text                                                               |
| Host github.com<br>HostName ssh.github.com<br>User git<br>Port 443 |

#### 2.1.2 Linux/macOS 系统：

bash

#创建或编辑 SSH 配置文件

```
nano ~/.ssh/config
```

添加以下内容：

```
text
```

```
Host github.com
```

```
    HostName ssh.github.com
```

```
    User git
```

```
    Port 443
```

## 2.2 验证配置是否生效

表 2-2 验证连接是否成功

|                       |
|-----------------------|
| bash                  |
| ssh -T git@github.com |

首次连接时会提示是否信任主机指纹，输入 yes 确认。如果配置成功，会显示认证成功的信息。

## 2.3 （可选）配置 Git 全局使用 443 端口

也可以直接用 Git 命令配置（2.3 和 2.2 只用选择一个做，注意用 2.3 后要去文件夹看一下如果有 txt 后缀需要删除）：

表 2-3 利用 git 命令配置

|                                                                                         |
|-----------------------------------------------------------------------------------------|
| bash                                                                                    |
| git config --global url."ssh://git@ssh.github.com:443".insteadOf "ssh://git@github.com" |

## 2.4 SSH 与 HTTPS 地址切换

（1）什么时候需要切换？

你配好了 SSH 密钥，平时用 SSH 地址推送，但偶尔需要临时用 HTTPS 地址拉取代码（比如在公司代理环境下，443 端口被白名单放行，SSH 不通）。

或者你一开始用 HTTPS 克隆的仓库，后来配好了 SSH 密钥，想改成 SSH 免密推送。

（2）两种地址长什么样？

## Git 基础教程

HTTPS 地址: `https://github.com/用户名/仓库名.git`

SSH 地址: `git@github.com:用户名/仓库名.git`

切换命令（二选一）：

表 2-4 SSH 和 HTTP 互相转化

|                                                                       |
|-----------------------------------------------------------------------|
| bash                                                                  |
| #切换为 HTTPS 地址（需要输入账号密码/Token）                                         |
| <code>git remote set-url origin https://github.com/用户名/仓库名.git</code> |
| #切换为 SSH 地址（免密推送，前提是已配好 SSH 密钥）                                       |
| <code>git remote set-url origin git@github.com:用户名/仓库名.git</code>     |
| 执行后确认是否切换成功：                                                          |
| bash                                                                  |
| <code>git remote -v</code>                                            |

会显示当前绑定的 `fetch` 和 `push` 地址，确认是你切换后的那个格式就行。

补充：如果切换后推送报错怎么办？

切到 HTTPS 后推送报错 `fatal: Authentication failed`→说明你没输对账号密码，GitHub 现在不支持密码登录，需要去 GitHub Settings→Developer settings→Personal access tokens 生成一个 Token，用 Token 当密码输入。

切到 SSH 后推送报错 `Permission denied (publickey)`→说明 SSH 密钥没配好或没加到 GitHub 上，回到第 1.3~1.5 节重新过一遍。

### 3 GitHub 仓库共享与拉取

GitHub 是云端 Git 服务器平台，专门存放远程共享仓库，用来备份本地代码、多人协作开发、下载开源项目源码。

一句话区分：Git 是本地管理工具，GitHub 是线上存放代码的服务器，二者搭配使用。

#### 3.1 如何共享仓库给其他人（添加协作者）

- (1) 进入你的 GitHub 仓库页面
- (2) 点击 Settings→左侧边栏选择 Collaborators
- (3) 点击 Add people，输入对方的 GitHub 用户名或邮箱
- (4) 对方会收到邀请邮件，接受后即可获得推送权限
- (5) 公开仓库任何人都可以 git clone 拉取查看，但只有被添加为 Collaborator 的人才能推送修改。

#### 3.2 克隆（拉取）别人的仓库到本地——如何指定存放路径

git clone 命令的作用是：把 GitHub 云端的一个远程仓库，完整下载到你电脑硬盘上的一个文件夹里。

核心认知：克隆到你电脑上之后，它就是一个实实在在的文件夹（里面包含源代码和.git隐藏目录），而不是单个文件，也不是云端网页。

- (1) 基本命令格式

表 3-1 git 基本命令

|                                                                              |
|------------------------------------------------------------------------------|
| bash                                                                         |
| git clone 远程仓库地址 [本地目标文件夹路径]                                                 |
| 远程仓库地址：必填，在 GitHub 仓库页面点击绿色 Code 按钮，选择 SSH 地址（格式：git@github.com:用户名/仓库名.git） |
| [本地目标文件夹路径]：选填，用来指定代码存到你电脑的哪个位置、文件夹叫什么名字                                     |

- (2) 三种使用场景（附详细说明）

场景一：不指定路径（最常用，适合新手）

表 3-2 使用场景 1

|                                                                |
|----------------------------------------------------------------|
| bash                                                           |
| git clone git@github.com:某用户/某仓库.git                           |
| 存放位置：当前 Git Bash 所在的目录（即当前文件夹右键调出 bash 的位置）。                   |
| 文件夹名称：自动以远程仓库名命名。                                              |
| 验证：你可以先在 Git Bash 里输入 pwd 查看当前路径，克隆完成后，去该路径下就能看到有一个名为某仓库的新文件夹。 |

场景二：指定相对路径（存放在当前目录的子文件夹里）

表 3-3 使用场景 2

|                                                        |
|--------------------------------------------------------|
| bash                                                   |
| git clone git@github.com:某用户/某仓库.git 我的项目备份            |
| 存放位置：当前 Git Bash 所在目录下的我的项目备份子文件夹里。                    |
| 文件夹名称：使用你指定的名称（我的项目备份），而不是仓库原名。                        |
| 前提：我的项目备份这个文件夹可以不存在（Git 会自动创建），但如果已存在且不为空，Git 会报错拒绝克隆。 |

场景三：指定绝对路径（直接存放到 D 盘、桌面等任意位置）

表 3-4 使用场景 3

|                                                        |
|--------------------------------------------------------|
| bash                                                   |
| git clone git@github.com:某用户/某仓库.git D:/嵌入式代码/STM32 项目 |
| 存放位置：直接存到 D:/嵌入式代码/STM32 项目目录下，与 Git Bash 当前所在位置无关。    |
| 文件夹名称：你指定的最后一层文件夹名称（此处为 STM32 项目）。                     |
| 验证：克隆完成后，打开 D 盘的嵌入式代码文件夹，就能看到 STM32 项目 这个文件夹。          |

（3）路径书写规范（Windows 用户必看，否则报错）

在 Git Bash 里输入路径，必须遵循以下规则：

盘符用小写字母开头，加冒号，斜杠用正斜杠/（正确：D:/项目；错误：D:\项目，反斜杠会被识别为转义符）

C 盘要写成 /c/ 而不是 C:/（正确：/c/Users/你的用户名/Desktop；错误：C:/Users/...，部分旧版本 Git Bash 不识别）

必须写完整到具体的文件夹名称，不能只写盘符（正确：git clone 地址 D:/STM32；错误：git clone 地址 D:，Git 不知道文件夹叫什么名字）

（4）目标文件夹已存在时会发生什么？

目标文件夹不存在→Git 自动创建该文件夹，正常克隆。

目标文件夹存在且为空→正常克隆成功。

目标文件夹存在且不为空→报错拒绝克隆（防止覆盖你原有的文件）。

解决方法：清空该文件夹，或换一个不存在的文件夹名称。

（5）再次明确：本地是「文件夹」，不是「文件」

很多新手会混淆这个概念，这里用一句话说清楚：

GitHub 云端：是一个网站，上面显示的是远程仓库（网页界面）

git clone 之后：代码被下载到你的电脑的硬盘上，变成了一个普通的文件夹（你可以用资源管理器打开、复制、粘贴、删除里面的文件）

所以，正确的说法是：“存在 D 盘的一个文件夹里。”文件夹里面装着所有源代码和子目录。

（6）进阶技巧：只想拉取仓库里的某一个子文件夹（稀疏检出）

有些开源仓库体积很大（包含几百 MB 的编译产物、安装包），你只想要里面的 Drivers/驱动文件夹或 Docs/文档文件夹。这时可以用稀疏检出（sparse checkout）来实现：

表 3-5 只拉取一个文件夹

|                                                  |
|--------------------------------------------------|
| bash                                             |
| 1.先初始化一个空仓库                                      |
| mkdir 我的项目                                       |
| cd 我的项目                                          |
| git init                                         |
| # 2. 绑定远程仓库                                      |
| git remote add origin git@github.com:某用户/某仓库.git |
| # 3. 开启稀疏检出模式                                    |
| git config core.sparseCheckout true              |

|                                                                 |
|-----------------------------------------------------------------|
| # 4. 指定你想拉取的文件夹路径（写入配置文件）                                       |
| <code>echo "Drivers/" &gt;&gt; .git/info/sparse-checkout</code> |
| <code>echo "Docs/" &gt;&gt; .git/info/sparse-checkout</code>    |
| # 5. 拉取代码（只拉取 main 分支）                                          |
| <code>git pull origin main</code>                               |

执行完成后，本地我的项目文件夹里只会出现 Drivers 和 Docs 两个子文件夹，其他大文件不会被下载，非常省空间和时间。

## 4 全新文件夹首次上传 GitHub 完整流程（使用 main 分支）

注意：GitHub 新仓库默认主分支名为 main，而非旧版的 master。

### 4.1 GitHub 网页端新建空仓库

登录 GitHub，点击右上角+→New repository

填写仓库名称，选择 Public 或 Private

不要勾选"Add a README file"（因为我们已有本地项目）

点击 Create repository

复制仓库的 SSH 地址（格式：git@github.com:你的用户名/仓库名.git）

### 4.2 本地初始化与首次上传

方法 1：进入本地项目文件夹，右键打开 Git Bash



图 4-1 git 打开方式

方法 2：这里在讲解一个在桌面打开 git bash 进入项目文件夹拉取推送的方法：

Windows 左下角搜索「Git Bash」打开终端，默认路径为用户家目录；

使用 cd 命令切换到你的项目文件夹，Git Bash 路径规则必须遵守：

Windows 盘符写法：D 盘写/d/、C 盘写/c/，斜杠统一用正斜杠/，禁止\；

路径带空格需用双引号包裹；

表 4-1 利用 cd 命令进入文件夹

|                                    |
|------------------------------------|
| #示例 1：进入 D 盘 STM32 学习项目（绝对路径）      |
| cd /d/stm32f103c8t6_learning_board |
| #示例 2：路径含空格，加引号                    |
| cd "/d/嵌入式代码/STM32 项目"             |
| #辅助目录操作命令                          |
| pwd # 查看当前所在文件夹路径                  |
| ls # 列出当前文件夹内所有文件                  |
| cd .. # 返回上一级文件夹                   |
| cd ~ # 回到默认用户根目录                   |

（1）初始化本地 Git 仓库（本地仓库仅创建第一次使用）：

表 4-2 git 初始化

|          |
|----------|
| Bash     |
| git init |

（2）绑定本地仓库与 GitHub 远程仓库（本地仓库仅创建第一次使用）：

表 4-3 绑定远程操作

|                                                      |
|------------------------------------------------------|
| Bash                                                 |
| git remote add origin git@github.com:你的用户名/你的仓库名.git |

（3）配置.gitignore 过滤无用文件（必须！）

表 4-4 .gitignore 过滤法

|                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>在项目根目录新建.gitignore 文件，写入过滤规则。这个文件告诉 Git 哪些文件/文件夹不要上传。</p> <p>语法规则详解：</p> <p>文件夹名/→排除项目内所有位置的该名称文件夹（如 temp/会排除根目录 temp、子目录里的 temp、任意深度的 temp）</p> <p>/文件夹名/→以斜杠开头，只排除项目根目录下的该文件夹（如/temp/只排除根目录的 temp，不排除子目录里的 temp）</p> <p>*.扩展名→排除项目内所有位置的该后缀文件（如*.bin 排除所有 bin 文件）</p> <p>路径精确匹配：abc/cbd/→只排除项目根目录下 abc 文件夹里的 cbd 子文件夹，不影响其他位置的 cbd</p> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>通配符+路径：abc/*.txt→排除 abc 文件夹里所有 txt 文件，不影响其他位置的 txt</p> <p>abc/cbd/*.bin→排除 abc/cbd/目录下所有 bin 文件</p> <p>!取反→在排除规则中放行某项（如*.lib 排除所有 lib 文件，但!重要.lib 单独保留）</p> <p>通俗理解：没有斜杠的就是只看名字，有斜杠的就是按路径找。写完整路径就能精确命中，只写名字就是通杀全局。</p> |
| <p>#Keil 编译产物（文件夹）</p> <p>Objects/</p> <p>Listings/</p> <p>#编译生成的文件（通配符，每条独占一行）</p> <p>*.bin</p> <p>*.hex</p> <p>*.axf</p> <p>*.elf</p> <p>#第三方工具安装包</p> <p>软件包/</p>                                                      |

注意：.gitignore 应在第一次提交之前定义好。一旦文件被提交了，再想清除就很麻烦。.gitignore 文件本身也需要被 Git 追踪，它会在下一步执行 git add.时被自动添加，无需单独执行 git add .gitignore。这样别人克隆你的仓库时，也能继承相同的过滤规则。

（4）添加所有需要上传的文件到暂存区：

表 4-5 上传暂存区

|           |
|-----------|
| Bash      |
| git add . |

生成本地版本，填写提交备注：

表 4-6 生成本地版本并注释

|                                      |
|--------------------------------------|
| bash                                 |
| git commit -m "首次上传：项目全套源码、原理图、说明文档" |

首次推送到 GitHub 远程仓库（使用 main 分支）：

表 4-7 首次推送命令

|                         |
|-------------------------|
| bash                    |
| git push -u origin main |

-u 参数会将本地 main 分支与远程 main 分支建立关联，之后直接输入 git push 即可。

仅第一次上传需要写 git push -u origin main，后续上传 git push 即可。

（5）如果远程仓库已有初始文件导致冲突，可强制覆盖：

表 4-8 强制推送

|                                 |
|---------------------------------|
| bash                            |
| git push -u origin main --force |

注：多人协作特别提醒：每次写代码前先拉取

如果这个仓库有其他人一起协作（比如你邀请了协作者，或者你克隆的是别人的共享仓库），绝对不能上来就直接写代码然后 git push。标准安全流程如下：

每天开工第一件事：打开 Git Bash，执行 git pull，把别人昨天或刚才推送的最新代码拉到你本地。

写代码、改代码、删文件（随便折腾）。

准备提交之前，再执行一次 git pull：防止在你写代码的过程中，别人已经先你一步推送了新内容。

最后执行三步走：git add .→git commit -m "备注"→git push。

为什么要“提交前再拉一次”？因为如果你写代码的这段时间里，别人已经推送了新的 commit 到远程仓库，而你本地没有，直接 git push 会被 GitHub 拒绝（报错 rejected）。先 git pull 把别人的更新合并到你本地，再推送，就能确保你的提交是基于最新版本的，不会覆盖别人的劳动成果。

### 4.3 分支搞错了怎么办？（master→main 迁移）

场景说明：你在本地初始化 Git 时，默认分支名为 master（旧版 Git 的习惯），但 GitHub 新仓库默认主分支是 main。结果你一直在 master 分支上开发和推送，远程仓库里同时存在 main 和 master 两个分支，main 是空的或只有初始文件，真正的代码全在 master 里。

目标：把 master 的代码完整转移到 main，然后删掉 master，以后统一用 main。

## Git 基础教程

(1) 先确认当前状态，确保所有改动已提交：

表 4-9 确认提交状态

|            |
|------------|
| bash       |
| git status |

确保显示 `nothing to commit, working tree clean`。

(2) 切换到 `master` 分支：

表 4-10 切换分支

|                     |
|---------------------|
| bash                |
| git checkout master |

(3) 强制让本地的 `main` 分支指向 `master` 的最新提交（把代码继承过来）：

表 4-11 主分支指向从分支

|                           |
|---------------------------|
| bash                      |
| git branch -f main master |

这条命令会把 `master` 的所有代码和历史记录完整复制给 `main`。如果本地没有 `main` 分支，它会自动新建一个。

(4) 切换到 `main` 分支：

表 4-12 切换回主分支

|                   |
|-------------------|
| bash              |
| git checkout main |

(5) 强制推送到远程仓库（覆盖远程的旧 `main`）：

表 4-13 推送覆盖旧主分支

|                         |
|-------------------------|
| bash                    |
| git push -f origin main |

⚠ 警告：-f 会覆盖远程 `main` 分支。仅在确认该仓库只有你一个人在维护 `main` 时使用。如果是多人协作，请先与团队成员沟通。

(6) 删除远程的 `master` 分支：

表 4-14 删除多余分支

|                                 |
|---------------------------------|
| bash                            |
| git push origin --delete master |

(7) 删除本地的 `master` 分支：

|                      |
|----------------------|
| bash                 |
| git branch -d master |

(8) 验证结果:

执行 `git branch`, 应该只看到 `main` 一个本地分支。

打开 GitHub 网页, 刷新仓库页面, 确认只有 `main` 分支, 且代码完整。

大功告成! 以后你的本地默认分支就是 `main`, 直接 `git push` 就行, 不会再搞混了。

补充: 另一种更简单的情况 (远程只有 `master`, 没有 `main`)

如果远程仓库里根本没有 `main` 分支 (只有 `master`), 直接重命名即可, 一步到位:

表 4-15 没有主分支情况

|                                              |
|----------------------------------------------|
| <code>bash</code>                            |
| <code>git branch -m master main</code>       |
| <code>git push -u origin main</code>         |
| <code>git push origin --delete master</code> |

然后去 GitHub 仓库的 `Settings`→`Branches`, 把默认分支从 `master` 改为 `main`。

## 5 日常更新简化流程（三步走）

### 5.1 Git 三大区域

Git 把文件变化分为三层，理解这三层就理解了所有 Git 命令的逻辑：

工作区：你电脑上能看到的项目文件夹，写代码、改文件都在这里。

暂存区：执行 `git add .` 之后，改动临时存放在这里，还没生成正式版本。

版本库：执行 `git commit -m "..."` 之后，改动永久保存为历史版本，存在 .git 文件夹里。

### 5.2 文件流转

首次绑定仓库、推送成功后，后续修改代码、新增文件仅需 3 条命令：

表 5-1 文件流转命令表

|                                                                 |
|-----------------------------------------------------------------|
| bash                                                            |
| #1. 抓取本次所有修改、新增、删除的文件                                           |
| <code>git add .</code>                                          |
| #2. 生成本地新版本，备注写清楚本次改动内容                                         |
| <code>git commit -m "更新：新增 OLED 屏幕驱动例程，修正串口发送 bug"</code>       |
| #3. 一键推送到 GitHub 云端                                             |
| <code>git push</code>                                           |
| #4. 只添加单个文件（精准控制，适合只改了一个文件）                                     |
| <code>git add main.c</code> （main.c 不是特指，你想添加哪个文件就写哪个）          |
| #5. 添加指定整个文件夹（递归包含所有子文件夹，嵌入式常用）                                 |
| <code>git add Drivers/</code> （同理不是特指，想添加哪个文件夹写哪个）              |
| #6. 一次性添加多个指定文件夹/文件，空格隔开                                        |
| <code>git add Drivers/ User/ README.md</code>                   |
| #7. 通配符添加同类型文件（只添加所有 .c 源码）*不包含以 . 开头的隐藏文件（如 .gitignore），而 . 包含 |
| <code>git add *.c</code>                                        |
| #8. 撤销全部已 add 的文件，退回工作区（修改保留，不会删除代码）                            |

|                                 |
|---------------------------------|
| <code>git reset .</code>        |
| #9.只撤销单个文件/单个文件夹的暂存             |
| <code>git reset main.c</code>   |
| <code>git reset Drivers/</code> |

如果只是新建子文件夹、放入新代码，无需任何额外初始化，Git 会自动识别新增的文件夹和文件，直接执行最上面三步即可。

⚠ 多人协作仓库：在执行下面三步之前，务必先执行 `git pull` 拉取别人最新的更新（详见第 4 章末尾的“多人协作特别提醒”）。

### 5.3 删除文件/文件夹的正确操作

删除文件分两种情况，操作不同，新手容易搞混：

情况一：彻底删除（本地和云端都删）

直接在文件夹里把文件/文件夹删掉，然后执行日常三步走即可：

表 5-2 本地删除法

|                                          |
|------------------------------------------|
| <code>bash</code>                        |
| <code>git add .</code>                   |
| <code>git commit -m "删除了某某文件/文件夹"</code> |
| <code>git push</code>                    |

Git 会自动识别“文件消失了”这个变化，把它当成一次删除记录，推送到云端后远程仓库里的对应文件也会被删除。

情况二：只删云端，保留本地文件（误传了大文件或敏感文件时用）

如果某个文件已经提交了，但你只想把它从 GitHub 上移除，本地电脑保留不动，用这个命令：

表 5-3 只删除云端法

|                                                |
|------------------------------------------------|
| <code>bash</code>                              |
| <code>git rm --cached 文件名/文件夹名</code>          |
| <code>git commit -m "从版本库中移除某某文件（本地保留）"</code> |
| <code>git push</code>                          |

`--cached` 参数的意思就是“只删版本库里的记录，不动硬盘上的源文件”，适合清理 `.gitignore` 没拦住的大文件、安装包、临时文件。

补充提醒：

删除文件夹时，如果文件夹里有内容，需要加-r 参数递归删除：`git rm -r --cached 文件夹名/`

如果只是想从版本库里删掉，但本地又不想保留，直接用 `git rm 文件名（没有--cached）` 即可（慎用），本地和云端一起删。

删除操作和新增、修改操作一样，都需要走 `add→commit→push` 三步，不会自动同步。

## 6 Git 常用操作命令速查

### 6.1 查看类命令

表 6-1 git 查看类命令

| 命令                   | 作用                                                       |
|----------------------|----------------------------------------------------------|
| git status           | 查看当前仓库状态：哪些文件被修改、待 add、待 commit（红色：未加入暂存区，绿色：已 add 到暂存区） |
| git log              | 按时间倒序显示所有提交历史（按 q 退出）                                    |
| git log --oneline    | 简化版，一行显示一个版本                                             |
| git diff             | 查看工作区所有未 add 的改动                                         |
| git diff --cached    | 查看暂存区所有已 add 的改动                                         |
| git diff 版本号 1 版本号 2 | 对比两个版本的差异                                                |
| git ls-files         | 列出所有已被 Git 追踪的文件                                         |
| git remote -v        | 查看当前绑定的远程仓库地址                                            |

### 6.2 分支管理命令

表 6-2 git 管理类命令

| 命令                | 作用          |
|-------------------|-------------|
| git branch        | 查看所有本地分支    |
| git branch 新分支名   | 新建分支        |
| git checkout 分支名  | 切换到指定分支     |
| git merge 分支名     | 合并指定分支到当前分支 |
| git branch -d 分支名 | 删除分支        |

git merge 提示冲突时，手动改文件→git add .→git commit 完成合并

## 6.3 远程仓库操作

表 6-3 git 远程操作命令

| 命令                                 | 作用                     |
|------------------------------------|------------------------|
| git remote set-url origin<br>新仓库地址 | 修改远程仓库地址               |
| git pull origin main               | 拉取远程仓库的最新代码并合并         |
| git clone 仓库地址                     | 克隆别人的仓库到本地             |
| git push                           | 推送本地提交到远程仓库（已绑定分支后直接用） |

## 6.4 版本回退

表 6-4 版本回退命令

| 命令                     | 作用                   |
|------------------------|----------------------|
| git reset --soft HEAD^ | 回退到上一个版本，保留改动文件      |
| git reset --hard HEAD^ | 彻底回退到上一个版本，丢弃所有改动    |
| git reset --hard 版本号   | 回退到指定版本（版本号填前 7 位即可） |

注意：--hard 会直接删除未提交的改动，执行前务必确认不需要保留当前代码。

## 6.5 临时保存（stash）

表 6-5 临时保存命令

| 命令             | 作用                   |
|----------------|----------------------|
| git stash      | 临时保存当前未提交的改动，还原干净工作区 |
| git stash list | 查看所有临时保存的记录          |
| git stash pop  | 恢复最近一次临时保存的改动        |

适用场景：写到一半代码，需要切换分支修 bug，又不想提交半成品代码。

git stash pop 后如果有冲突，手动解决后执行 git add .和 git stash drop 删除 stash 记录

## 6.6 文件操作

表 6-6 文件操作命令

| 命令                               | 作用               |
|----------------------------------|------------------|
| <code>git rm --cached 文件名</code> | 从版本库中删除文件，本地文件保留 |
| <code>git rm 文件名</code>          | 从版本库和本地同时删除文件    |
| <code>git mv 旧文件名 新文件名</code>    | 重命名文件，同时保留版本历史   |

## 7 常见问题与避坑指南

### 7.1 推送报错：文件超过 100MB

GitHub 单文件硬性上限 100MB。解决方法：

用 .gitignore 屏蔽该文件

用 `git rm --cached` 从版本库中移除该文件

大文件单独存网盘

### 7.2 推送报错：Connection timed out port 22

原因：22 端口被防火墙封锁。

解决方法：按第 2 章方法将 SSH 端口改为 443。

### 7.3 不要嵌套初始化 Git 仓库

一个项目文件夹只能执行一次 `git init`，不要在子文件夹里再执行 `git init`，会导致版本管理混乱。

### 7.4 提交备注规范

备注一定要写清楚本次改动内容，方便以后回溯。

✅ 好的示例："修复串口接收丢包 bug，新增 ADC 采样例程"

❌ 坏的示例："更新"、"改了点东西"

### 7.5 .gitignore 不生效怎么办？

如果某些文件已被 Git 追踪，后来才添加到 .gitignore，需要手动取消追踪：

表 7-1 删除之前上传文件命令

|                                  |
|----------------------------------|
| bash                             |
| <code>git rm --cached</code> 文件名 |

## 7.6 多人协作时严禁使用--force 强制推送

(1) 先讲一个场景，你就明白为什么了：

假设你和同事小王在同一个仓库里协作。你们俩本地都克隆了同一个仓库，远程仓库里有一个 `main.c` 文件，内容是：`int version = 1;`

上午 9:00：小王改了代码，把 `version` 改成 2，然后执行了 `git add .`、`git commit`、`git push`。此时远程仓库的 `main.c` 变成了：`int version = 2;`

上午 9:30：你也在本地改了代码，但你忘了先 `git pull`。你直接执行了 `git add .` 和 `git commit`，然后准备推送。

此时你的本地 `main.c` 还是基于旧版本（`version = 1`）改的，比如你改成了：`int version = 3;`

如果你用 `git push`（正常推送）：GitHub 会报错 `rejected`，告诉你“远程有更新，你先拉下来合并”。这是保护机制，防止你覆盖别人的代码。你执行 `git pull` 后，Git 会把小王的改动（`version = 2`）和你本地的改动（`version = 3`）合并在一起，如果冲突就手动解决，最后推上去的代码是 `version = 3`，同时小王的 `version = 2` 也被保留在历史记录里。

但如果你用 `git push --force`（强制推送）：GitHub 会直接把你的本地版本完整覆盖到远程仓库，完全跳过“检查远程是否有新提交”的步骤。小王的 `version = 2` 会在远程仓库里彻底消失，就像他从来没提交过一样。小王下午一拉代码，发现自己上午的工作不见了，只能痛苦地重写或者从本地找备份。

(2) 为什么会丢失？

因为 Git 的历史记录是一条链（每个 `commit` 都指向前一个 `commit`）。强制推送的本质是“把远程的指针强行挪到你的最新提交上”，中间别人提交的那些 `commit` 如果不在你的本地分支上，就会变成“孤儿节点”，Git 过段时间会自动清理掉它们。不是被覆盖，而是被彻底丢弃。

正确的做法是什么？

表 7-2 多人协作推送失败正确修改方法

|                       |
|-----------------------|
| <code>bash</code>     |
| #推送被拒后，先拉取别人的更新       |
| <code>git pull</code> |
| #如果有冲突，手动解决冲突后保存      |

|                          |
|--------------------------|
| #再重新 add → commit → push |
| git add .                |
| git commit -m "合并了小王的代码" |
| git push                 |

安全规则：

✅ 单人仓库、确信无人协作时→可用--force

❌ 多人协作仓库→绝对禁用--force

如果你想在多人协作仓库里覆盖远程分支，有更安全的替代命令：`git push --force-with-lease`。它的作用是“只有当远程分支和你上次拉取时一模一样时，才允许覆盖”，能防止误伤别人的提交。但新手阶段不建议碰任何带--force 的命令，先学会老老实实 `git pull` 再 `git push`。

## 7.7 提交备注写错了怎么改？

情况 1：如果刚执行完 `git commit -m "写错了"`，发现备注写错了，且还没有执行 `git push`，可以用以下命令修改最近一次 commit 的备注：

表 7-3 修改备注方法

|                                |
|--------------------------------|
| bash                           |
| git commit --amend -m "新的正确备注" |

这条命令会直接替换掉上一次 commit 的备注信息，不会产生新的提交记录，历史记录里看起来就像一开始就写对了一样。

⚠️ 注意：如果已经执行了 `git push` 推送到远程仓库，就不要用--amend 了，因为远程仓库已经记录了旧备注，强行修改会导致版本不一致。正确的做法是重新提交一次，写清楚改正了什么。

情况 2：如果备注写错了，且已经执行了 `git push`，不能直接用--amend 改本地备注再 push——因为远程仓库已经有了旧备注的提交记录，你的本地备注和远程备注不一致，Git 会报错拒绝推送。

正确操作分两步：

(1) 修改本地最近一次提交的备注：

表 7-4 修改本地备注

|      |
|------|
| bash |
|------|

```
git commit --amend -m "新的正确备注"
```

(2) 强制推送到远程仓库（覆盖远程的旧提交）：

表 7-5 强制推送覆盖旧的

```
bash
```

```
git push --force-with-lease origin main
```

这里用的是`--force-with-lease`，而不是`--force`。区别在于：`--force-with-lease`会先检查远程分支是否和你上次拉取时一致，只有确认没人在这期间推送过新提交，才会执行覆盖。比裸`--force` 安全得多。

⚠ 重要前提：

这个仓库只有你一个人在维护时，才适合用这个方法改备注。

如果是多人协作仓库，不要改已经 `push` 的备注。因为其他人本地可能已经拉取了旧备注的提交，你强行改掉会导致他们本地历史和你云端历史不一致，合并时会非常混乱。正确的做法是再提交一次，备注写“修正备注：.....”来补充说明。